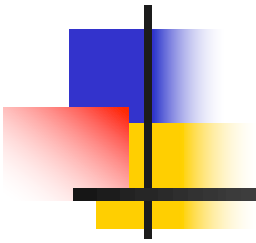
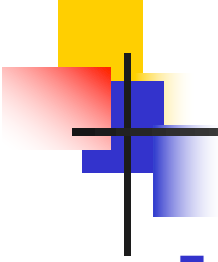


Tema 3: Capa Lógica de Negocio con Spring

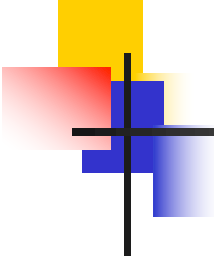


- Introducción
- Inyección de dependencias
- Transaccionalidad
- Optimistic Locking
- Pruebas automatizadas



Introducción

- La capa Lógica de Negocio de PA Shop ofrece 14 casos de uso, agrupados en 3 servicios locales (fachadas)
 - **UserService**: registro de usuarios
 - **CatalogService**: búsqueda de productos
 - **ShoppingService**: gestión del carrito, compra y visualizados de pedidos
- La implementación de estos servicios hace uso de un servicio interno que no se expone a la capa superior
 - **PermissionChecker**: verificación de permisos de acceso (e.g. un usuario sólo puede acceder a su propio carrito)

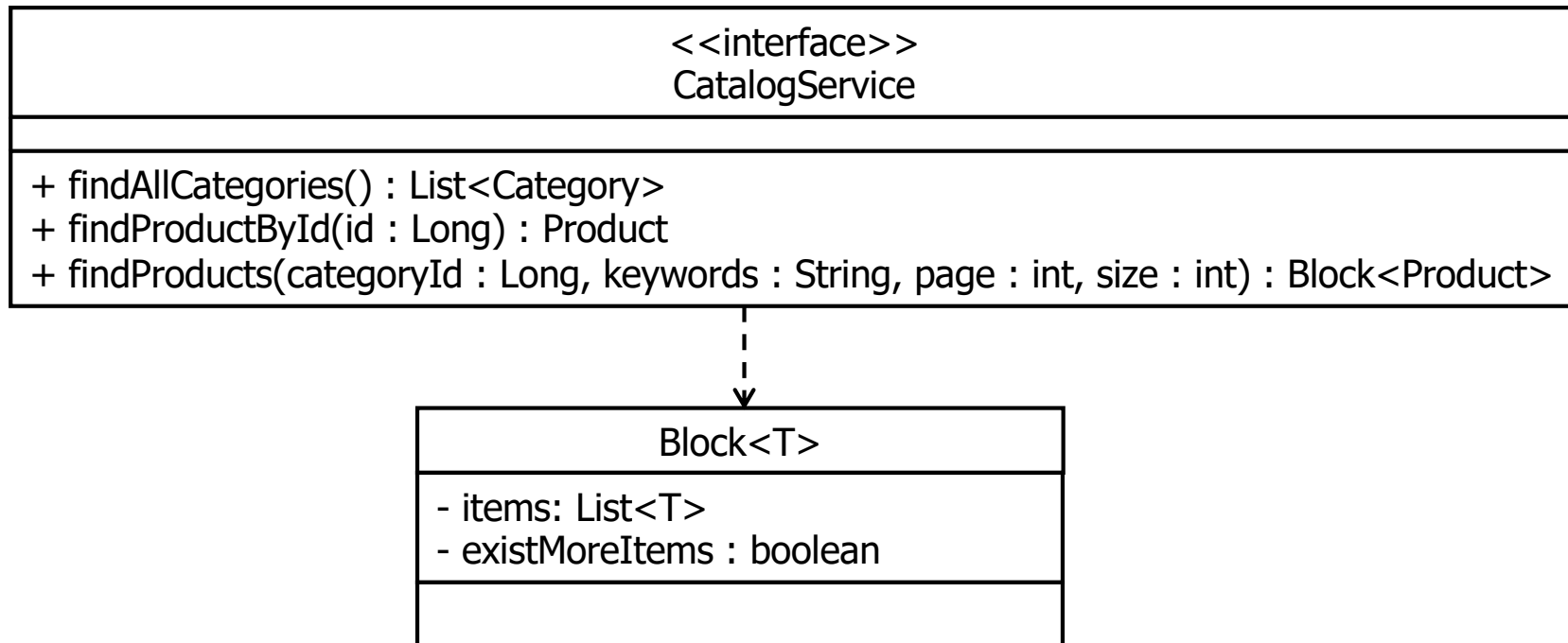


Introducción

<<interface>> UserService	
+ signUp(User user) : void + login(userName: String, password : String) : User + loginFromId(id : Long): User + updateProfile(id : Long, firstName : String, lastName : String, email : String) : User + changePassword(id : Long, oldPassword : String, newPassword : String) : void	

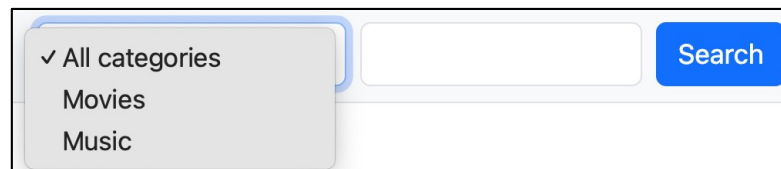
- **signUp**: el parámetro **user** es de entrada/salida
 - Después de su ejecución, **user.getId()** devolverá el identificador
- **loginFromId**: se explica como parte del frontend (tema 7)
- Tras invocar **signUp**, **login** o **updateProfile**, el frontend cachea la información del usuario, excepto la contraseña
 - Cuando el usuario accede a su perfil, éste se recupera de la caché
- **updateProfile** y **changePassword**: tanto en este servicio, como en **ShoppingService**, la capa Servicios REST (tema 4) impide que un usuario pase un identificador de usuario (parámetro **id**) arbitrario

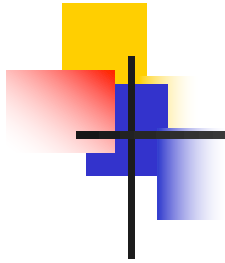
Introducción



■ **findAllCategories**

- Necesaria para mostrar el desplegable de categorías
- No tiene sentido usar paginación
- El frontend cacheará el resultado devuelto





Introducción

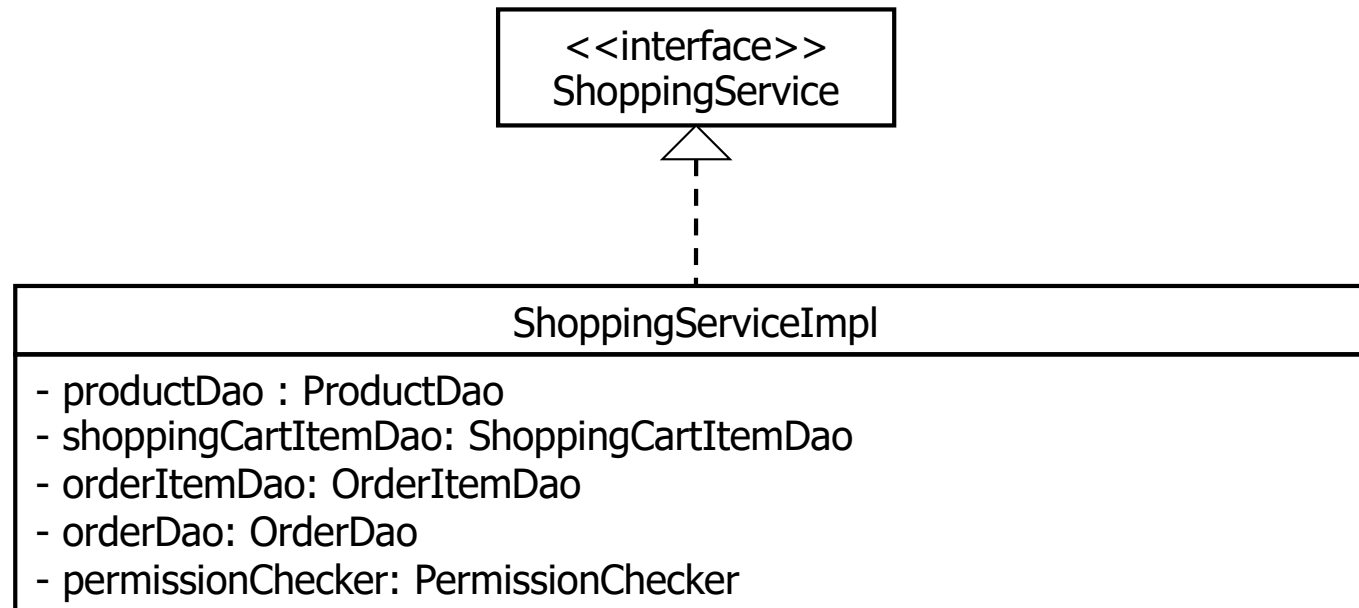
<<interface>>
ShoppingService

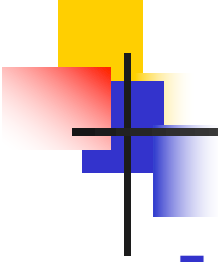
```
+ addToShoppingCart(userId: Long, shoppingCartId: Long, productId: Long, quantity: int) : ShoppingCart
+ updateShoppingCartItemQuantity(userId: Long, shoppingCartId: Long, productId: Long, quantity: int) :
  ShoppingCart
+ removeShoppingCartItem(userId: Long, shoppingCartId: Long, productId: Long) : ShoppingCart
+ buy(userId: Long, shoppingCartId: Long, postalAddress: String, postalCode: String) : Order
+ findOrder(userId: Long, orderId: Long) : Order
+ findOrders(userId: Long, page: int, size: int) : Block<Order>
```

- Tras invocar **addToShoppingCart**, **updateShoppingCartItemQuantity** o **removeShoppingCartItem**, el frontend cachea la información del carrito
 - Cuando el usuario visualiza su carrito, éste se recupera de la caché

Inyección de dependencias

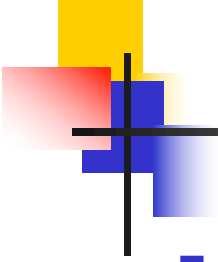
- La implementación de los servicios locales necesita hacer uso de
 - Los DAOs necesarios
 - El servicio interno **PermissionChecker**
- Ejemplo





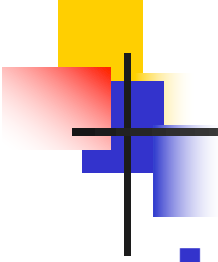
Inyección de dependencias

- La implementación de cada servicio debe poder obtener una instancia de cada DAO y el servicio interno sin necesidad de conocer las clases de implementación
- Una primera aproximación consistiría en definir una factoría por cada DAO y el servicio interno
 - Sin embargo, en este tipo de situaciones es demasiado tedioso (muchos DAOs en un caso real)
- Los frameworks profesionales permiten resolver esta situación mediante el paradigma de la **Inyección de Dependencias**
 - <https://martinfowler.com/articles/injection.html>



Inyección de dependencias

- Bajo el paradigma de la inyección de dependencias, existe un **contenedor de objetos** que [1] **crea** los objetos y [2] les **inyecta** las referencias a los objetos de los que dependen
 - Normalmente, la creación de objetos y la inyección ocurre durante el arranque de la aplicación
- En general, este paradigma no se aplica a todos los objetos de una aplicación/servicio, sino sólo a un pequeño subconjunto
- **En la capa Modelo, aplicaremos este paradigma a los DAOs y los servicios locales**



Inyección de dependencias

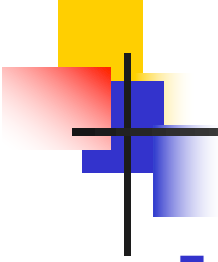
- ¿Cómo se usa la inyección de dependencias en Spring?
 - Dos opciones: ficheros de configuración o anotaciones
 - Usaremos **anotaciones**

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
...
@Service
public class ShoppingServiceImpl implements ShoppingService {

    @Autowired
    private PermissionChecker permissionChecker;

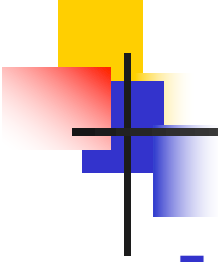
    @Autowired
    private ProductDao productDao;

    @Autowired
    private ShoppingCartItemDao shoppingCartItemDao;
    ...
}
```



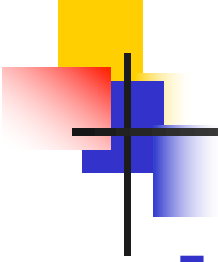
Inyección de dependencias

- [1] Especificación de clases que gestionará el contenedor de objetos
 - NOTA: en Spring, a los objetos gestionados por su contenedor se les llama **beans**
 - **@Repository**: para DAOs
 - En nuestro caso es implícita, dado que los DAOs los implementa automáticamente Spring Data y automáticamente son beans
 - **@Service**: para servicios locales
- [2] Inyección de dependencias
 - **@Autowired**
 - Por defecto, inyecta un bean que implemente la interfaz del atributo
 - Si hay más de un bean que implemente esa interfaz, es necesario especificar cuál mediante el uso de otras anotaciones



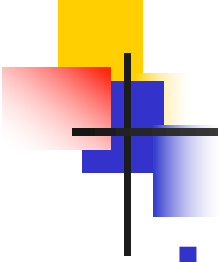
Inyección de dependencias

- **Ámbito de los beans**
 - Por defecto, los beans son “singletons”
 - Una única instancia de cada DAO y una única instancia de cada servicio local
- **Durante el arranque del backend**
 - [1] El contenedor de objetos de Spring inspecciona las clases anotadas explícita o implícitamente con `@Repository` y `@Service` y crea una instancia de cada una (bean)
 - [2] Para cada bean creado, inspecciona los atributos anotados con `@Autowired` y los inicializa con un bean de ese tipo



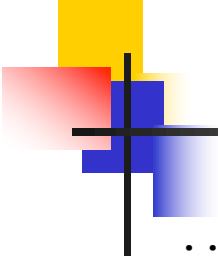
Transaccionalidad

- Spring permite usar un **enfoque declarativo** para implementar la transaccionalidad de los métodos de los servicios locales
- Es posible especificar la transaccionalidad mediante ficheros de configuración o mediante la anotación **@Transactional**
- La anotación **@Transactional** se puede declarar a nivel de clase o método
 - A nivel de clase aplica a todos los métodos públicos
 - A nivel de método sobrescribe la semántica especificada a nivel de clase



Semántica por defecto de @Transactional

- **[ANTES]** Cuando se invoca a un método anotado (directa o indirectamente) con **@Transactional**
 - Si se hace fuera del contexto de una transacción, se crea una nueva
 - Si se hace dentro del contexto de una transacción, se engancha a ella
- **[EJECUCIÓN DEL MÉTODO]** Las operaciones que se invoquen de los DAOs/servicios se enganchan a la transacción
- **[DESPUÉS]** Commit/rollback
 - Si la ejecución del método no lanzó ninguna excepción, la transacción termina con **commit**
 - Si se lanza una excepción checked, la transacción termina con **commit**
 - Si se lanza una excepción de runtime o un **Error**, la transacción termina con **rollback**
- Internamente, la implementación de Spring intercepta las peticiones a los métodos anotados con **@Transactional** para implementar la semántica descrita



Transaccionalidad – Ejemplo de esquema de implementación

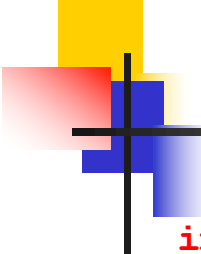
```
...
import org.springframework.transaction.annotation.Transactional;
...
@Service
@Transactional
public class <<Service>>Impl implements <<Service>> {

    ...

    @Override
    public ... <<useCase>> (...) throws ... {
        ...
    }

    @Override
    @Transactional(readonly=true)
    public ... <<readOnlyUseCase>> (...) throws ... {
        ...
    }
}
```

Si un método sólo ejecuta operaciones de lectura, se puede especificar `readonly=true` para permitir que el gestor de transacciones subyacente realice optimizaciones.



Transaccionalidad – Ejemplo

```
import org.springframework.transaction.annotation.Transactional;
```

```
...
```

```
@Service
```

```
@Transactional
```

```
public class ShoppingServiceImpl implements ShoppingService {
```

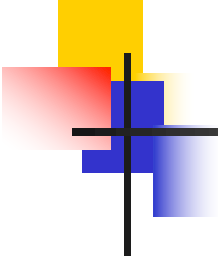
```
    ...
```

```
    @Override
```

```
    public Order buy(Long userId, Long shoppingCartId,  
        String postalAddress, String postalCode)  
        throws InstanceNotFoundException, PermissionException,  
        EmptyShoppingCartException {
```

```
        ShoppingCart shoppingCart = permissionChecker.  
            checkShoppingCartExistsAndBelongsTo(shoppingCartId,  
                userId);
```

```
        if (shoppingCart.isEmpty()) {  
            throw new EmptyShoppingCartException();  
        }
```

Transaccionalidad – Ejemplo

```
Order order = new Order(shoppingCart.getUser(),  
    LocalDateTime.now(), postalAddress, postalCode);  
orderDao.save(order);
```

```
for (ShoppingCartItem shoppingCartItem :  
    shoppingCart.getItems()) {
```

```
    OrderItem orderItem = new OrderItem(  
        shoppingCartItem.getProduct(),  
        shoppingCartItem.getProduct().getPrice(),  
        shoppingCartItem.getQuantity());
```

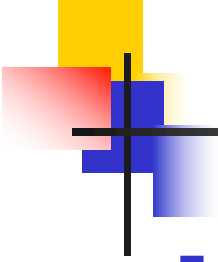
```
    order.addItem(orderItem);  
    orderItemDao.save(orderItem);  
    shoppingCartItemDao.delete(shoppingCartItem);
```

```
}
```

```
shoppingCart.removeAll();
```

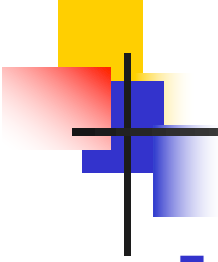
```
return order;
```

```
}
```



Transaccionalidad – Ejemplo

- Observación I: transaccionalidad
 - Cuando desde la capa de Servicios REST se invoca a **buy**, se crea una nueva transacción (**@Transactional** en **ShoppingServiceImpl**)
 - La invocación de **checkShoppingCartExistsAndBelongsTo** se engancha a la transacción actual (**PermissionCheckerImpl** usa **@Transactional** a nivel de clase)
 - Las operaciones invocadas de los DAOs se enganchan a la transaccional actual
 - En resumen, la ejecución de **buy** ocurre dentro de una única transacción



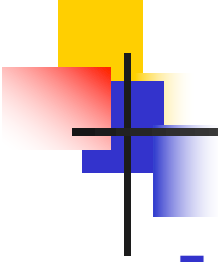
Transaccionalidad – Ejemplo

- Observación II: Domain Model

- Como se comentó en el tema 2, las entidades ofrecen métodos de negocio (patrón Domain Model) para facilitar la implementación de las capas Lógica de Negocio y Servicios
- `ShoppingServiceImpl.buy` hace uso de `ShoppingCart.{isEmpty, removeAll}` y `Order.addItem`
- En `ShoppingCart`

```
@Transient
public boolean isEmpty() {
    return items.isEmpty();
}

public void removeAll() {
    items = new HashSet<>();
}
```



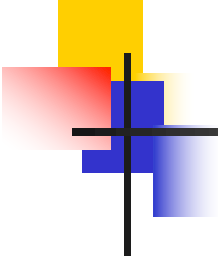
Transaccionalidad – Ejemplo

- Observación II : Domain Model

- En Order

```
public void addItem(OrderItem item) {  
  
    items.add(item);  
    item.setOrder(this);  
  
}
```

- **Pregunta: ¿qué ocurriría si
ShoppingServiceImpl.buy no invocase a
order.addItem(orderItem)?**

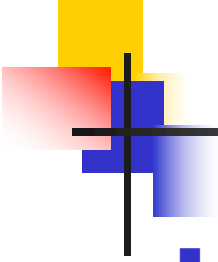


Transaccionalidad – Ejemplo

```
@Override
@Transactional(readonly=true)
public Order findOrder(Long userId, Long orderId)
    throws InstanceNotFoundException, PermissionException {

    return permissionChecker.checkOrderExistsAndBelongsTo(orderId,
        userId);
}

}
```



Transaccionalidad – entidades en estado “dirty”

- Si dentro de una transacción, se modifica una entidad en memoria (se dice que está en estado “dirty”), no es necesario invocar al método **save** del DAO
 - Hibernate lanzará la sentencia de actualización en BD automáticamente justo antes de terminar la transacción
- En **UserServiceImpl**

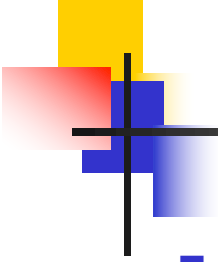
```
@Override
public User updateProfile(Long id, String firstName,
    String lastName, String email) throws InstanceNotFoundException {

    User user = permissionChecker.checkUser(id);

    user.setFirstName(firstName);
    user.setLastName(lastName);
    user.setEmail(email);

    return user;

}
```



Transaccionalidad – caché de primer nivel

- Las implementaciones de JPA hacen uso de **caché de primer nivel** (“first-level cache”)
 - Durante la ejecución de una transacción, el mapeador objeto-relacional cachea las instancias de las entidades con las que se trabaja durante esa transacción
 - Al final de la transacción, la caché se elimina
 - La caché no se comparte entre transacciones distintas
- Las implementaciones de JPA habitualmente también disponen de caché de segundo nivel (“second-level cache”)
 - Permiten cachear entidades y resultados de consultas a nivel de servicio
 - Requiere configuración

Transaccionalidad – caché de primer nivel: ejemplo

- Supongamos dos métodos de un servicio anotado con `@Transactional`, que hacen uso de las entidades `Product` y `Category` del caso de estudio

```
public void example1() {
```

```
    Category category = new Category(...);
```

```
    categoryDao.save(category); -----> INSERT + instancia cacheada
```

```
    category = categoryDao.findById(category.getId()).get();
```

```
}
```

```
-----> No provoca SELECT
```

```
public void example2(Long productId) {
```

```
    Product product = productDao.findById(productId).get();
```

```
    Category category = product.getCategory(); --> Proxy sin inicializar
```

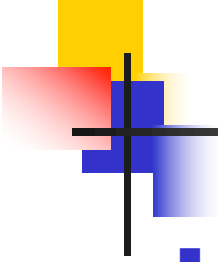
```
    Long categoryId = category.getId();
```

```
    String categoryName = category.getName(); --> SELECT + instancia cacheada
```

```
    category = categoryDao.findById(categoryId).get();
```

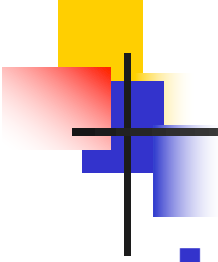
```
}
```

```
-----> No provoca SELECT
```

Optimistic Locking – Problema

- Con suficiente carga, trabajar con un nivel de aislamiento superior a **TRANSACTION_READ_COMMITED** no es escalable
 - La BD tendría que hacer muchos bloqueos en las filas afectadas por una transacción para garantizar la semántica de aislamiento
- Hibernate asume por defecto que no se utiliza un nivel de aislamiento superior a **TRANSACTION_READ_COMMITED**
- Consecuencias
 - De manera general, la transacción puede tener problemas de “second lost update”, “non-repeatable reads” y “phantom reads”
 - La mayor parte de las transacciones no relanzan consultas, y en consecuencia, no se verán afectadas por “non-repeatable reads” y “phantom reads”
 - Sin embargo, es habitual que una transacción tenga que leer una entidad de BD, actualizarla en memoria y finalmente en BD => se verá afectada por “second lost update”



Optimistic Locking – Problema

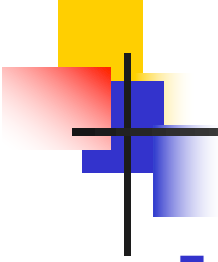
- Consideremos el siguiente caso de uso en un hipotético servicio local
 - Entre otros, la entidad **Account** dispone de los atributos/propiedades **id** y **balance**
 - **Account.add** suma la cantidad recibida como parámetro al balance

```
@Transactional
public class AccountServiceImpl implements AccountService {
    ...
    @Override
    public void add(Long accountId, BigDecimal amount)
        throws InstanceNotFoundException {

        Account account = findAccount(accountId);

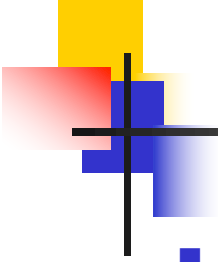
        account.add(amount);

    }
}
```



Optimistic Locking – Problema

- Supongamos que este caso de uso se puede invocar concurrentemente, e incluso con otros que también modifican el balance de una cuenta (e.g. **withdraw**, **transfer**)
- Para simplificar, consideremos la situación en la que dos empleados del banco (quizás en distintas oficinas), desean añadir 100€ y 200€ a la cuenta 1 (que inicialmente tenía 1000€), respectivamente, “justo” en el mismo momento
 - Inevitablemente, una comienza antes que la otra
 - Supongamos que la que comienza después lo hace antes de que la primera haga un commit
 - Inevitablemente, una termina antes que la otra



Optimistic Locking – Problema

- Ejemplo de ejecución
 - [T1] lee cuenta 1 [balance=1000]
 - [T2] lee cuenta 1 [balance=1000]
 - [T1] Suma 100 a **account** [balance=1100]
 - [T2] Suma 200 a **account** [balance=1200]
 - [T1] Actualiza **account** en BD y termina la transacción con commit
 - Supongamos que T1 termina antes que T2
 - UPDATE Account SET balance=1100, <<resto de columnas>> WHERE id=1
 - Commit
 - balance = 1100 en BD
 - [T2] Actualiza **account** en BD y termina la transacción con commit
 - UPDATE Account SET balance=1200, <<resto de columnas>> WHERE id=1
 - Commit
 - balance = 1200 en BD
- Second lost update
 - T2 sobrescribe la valor del balance establecido por T1

Optimistic Locking – Solución

- Es posible evitar el problema de “second lost update” con el nivel de aislamiento **TRANSACTION_READ_COMMITTED** mediante la técnica de **Optimistic Locking**
- En el caso particular de JPA, la solución consiste en
 - Añadir a la entidad un atributo/propiedad, habitualmente de tipo numérico, anotado con **@Version**

...

```
private long version;
```

```
@Version
```

```
public long getVersion() {  
    return version;  
}
```

```
public void setVersion(version) {  
    this.version = version;  
}
```

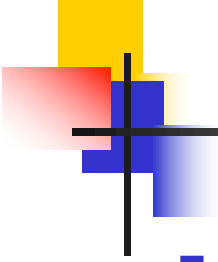
...

- Añadir la columna correspondiente a la tabla de la entidad

id	balance	...	version
----	---------	-----	---------

(PK)

Tabla Account



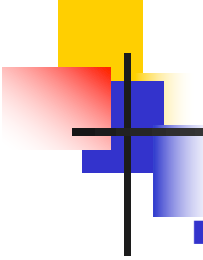
Optimistic Locking – Solución

- Funcionamiento

- El valor del atributo/propiedad anotado con `@Version` (e.g. `version`) lo establece automáticamente el mapeador objeto/relacional
- Cuando se inserta una instancia de una entidad en BD, se crea con `version=0`
- Cuando se recupera una instancia de una entidad de BD, se recuperan todas las columnas, inclusive `version`
- Cada vez que se actualiza una instancia de una entidad en BD se lanza una consulta del tipo

```
UPDATE Account SET <<columnas>>, version=version+1 WHERE  
id=<<id entidad>> AND version=<<valor de version en memoria>>
```

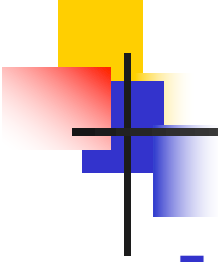
- Si el número de filas actualizadas es 0 => el mapeador lanza una excepción de runtime
 - RECORDATORIO: `PreparedStatement.executeUpdate` devuelve el número de filas que se vieron afectadas por la consulta



Optimistic Locking - Solución

■ El ejemplo anterior con Optimistic Locking

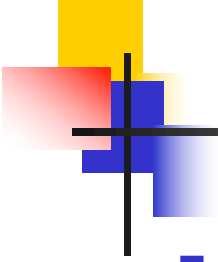
- Supongamos que en BD la cuenta con id 1 tiene balance = 1000 y version = 10
- [T1] lee cuenta 1 [balance=1000, version=10]
- [T2] lee cuenta 1 [balance=1000, version=10]
- [T1] Suma 100 a **account** [balance = 1100, version=10]
- [T2] Suma 200 a **account** [balance = 1200, version=10]
- [T1] Actualiza **account** en BD y termina la transacción con commit
 - Supongamos que T1 termina antes que T2
 - UPDATE Account SET balance=1100, <<resto de columnas>>, version=version+1 WHERE id=1 AND version=10
 - Commit
 - balance=1100, version=11 en BD
- [T2] Actualiza **account** en BD y termina la transacción con commit
 - UPDATE Account SET balance=1200, <<resto de columnas>>, version=version+1 WHERE id=1 AND version=10
 - Ninguna fila actualizada => excepción de runtime (mapeador) => rollback (Spring)
 - balance=1100, version=11 en BD



Optimistic Locking - Solución

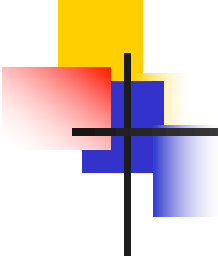
- Observaciones

- La estrategia escala porque no realiza ningún bloqueo sobre las filas que se leen durante la transacción
- Antes de terminar la transacción se incluye un chequeo para comprobar si hubo conflicto con otras transacciones (¿número de filas actualizadas = 0?)
- Si hay conflicto, la excepción de runtime se puede dejar fluir hacia arriba e informar al usuario de que hubo un error temporal y que la operación no se ejecutó (se hizo un rollback)



Pruebas automatizadas

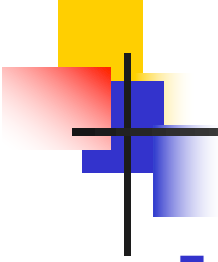
- Al igual que en ISD
 - Las pruebas automatizadas del servicio `<<Service>>`, se implementarán en la clase `<<Service>>Test`
 - Cada caso de prueba se implementará en un método `@Test`
 - Los casos de prueba deben ser independientes entre sí
 - Se utilizan DAOs desde las clases de pruebas cuando es necesario (aunque ahora será más cómodo)
 - El caso de estudio utiliza dos bases de datos
 - `pa`: para probar la aplicación como desarrollador
 - `patest`: para ejecutar las pruebas automatizadas



Pruebas automatizadas – Esquema de implementación

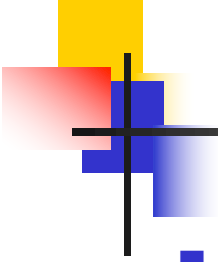
```
...
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.ActiveProfiles;
import org.springframework.transaction.annotation.Transactional;
import static org.junit.jupiter.api.Assertions.*;
...

@SpringBootTest
@ActiveProfiles("test")
@Transactional
public class <<Service>>Test {
    @Autowired
    private <<Service>> service;
    @Autowired
    private <<Dao>> dao;
    @Test
    public void test<<UseCase>>() throws ... {
        ...
        assertXxx(...);
    }
    ...
}
```



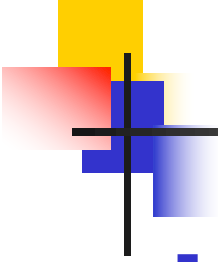
Pruebas automatizadas – spring-test

- Usaremos la librería **spring-test** (incluida transitivamente por spring-boot-starter-test) para implementar ágilmente las clases de pruebas de los servicios de la capa Lógica de Negocio
 - Por defecto, usa JUnit 5
- La anotación `@SpringBootTest` a nivel de clase permite
 - **Que las pruebas se ejecuten con JUnit 5**
 - **Hacer uso de las características propias de Spring Boot (e.g. acceder a su configuración)**
 - **Injectar beans (e.g. DAOs y servicios) en las clases de pruebas**



Pruebas automatizadas – spring-test

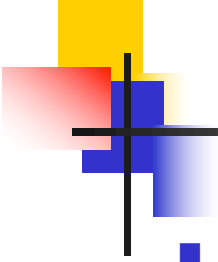
- Usaremos la anotación `@Transactional` a nivel de clase
 - Cada caso de prueba se ejecutará en una transacción, a la que se engancharán los métodos de los DAOs y servicios que invoque
- Con spring-test, por defecto, cada método anotado, directa o indirectamente con `@Transactional`, termina con rollback
 - **Las modificaciones que haya hecho el caso de prueba a la BD se deshacen automáticamente**
 - En consecuencia, cada caso de prueba no necesita eliminar los datos que crea



Pruebas automatizadas – Configuración

- Parte de la configuración no es adecuada para la ejecución de pruebas automatizadas

```
spring:
  datasource:
    url: @dataSource.url@ -----> jdbc:mysql://.../pa?...
    username: @dataSource.user@ -----> pa
    password: @dataSource.password@ -----> pa
    hikari:
      maximum-pool-size: 4
      ...
  jpa:
    hibernate:
      ddl-auto: none
  ...
```



Pruebas automatizadas – Configuración

- Además de la configuración general (`application.{yml, properties}`), Spring permite tener configuración específica a “perfiles”
- A menudo, el concepto de perfil se aplica a “entorno de desarrollo” (desarrollo, pruebas, producción, etc.)
- Con Spring Boot, si se desea tener configuración específica a un perfil, basta crear el fichero `application-{perfil}.{yml, properties}`
 - Estos ficheros pueden añadir o redefinir propiedades de la configuración general
- Cuando se ejecuta el backend, se puede especificar el perfil o perfiles que se desea que se consideren. Ejemplo:
 - `java -jar backend.jar --spring.profiles.active=production`

Pruebas automatizadas – Configuración

- En el caso particular de las pruebas automatizadas, se puede usar la anotación `@ActiveProfiles` (spring-test) para especificar los perfiles que se desea que se apliquen
- En el caso de estudio
 - Se usa `@ActiveProfiles` sobre las clases de pruebas de los servicios, especificando el perfil `test`
 - Se define `src/test/resources/application-test.yml`, redefiniendo las propiedades necesarias

spring:

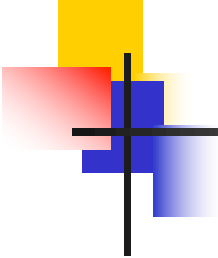
datasource:

url: @testDataSource.url@ -----> jdbc:mysql://.../patest?...
username: @testDataSource.user@ -----> pa
password: @testDataSource.password@ -----> pa

hikari:

maximum-pool-size: 1

En el caso de estudio las propiedades Maven `testDataSource.user` y `testDataSource.password` tienen el mismo valor que las propiedades `dataSource.user` y `dataSource.password`, pero lo correcto es redefinir `username` y `password` porque los valores de las propiedades Maven podrían ser distintos.



Pruebas automatizadas – Ejemplo

```
@SpringBootTest
```

```
@ActiveProfiles("test")
```

```
@Transactional
```

```
public class ShoppingServiceTest {
```

```
...      org.springframework.transaction.annotation.Transactional  
        y NO jakarta.transaction.Transactional
```

```
@Autowired
```

```
private UserService userService;
```

```
@Autowired
```

```
private ShoppingService shoppingService;
```

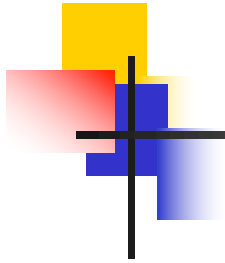
```
@Autowired
```

```
private CategoryDao categoryDao;
```

```
@Autowired
```

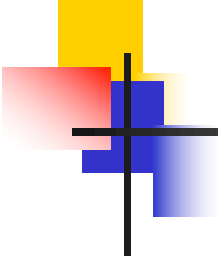
```
private ProductDao productDao;
```

```
...
```

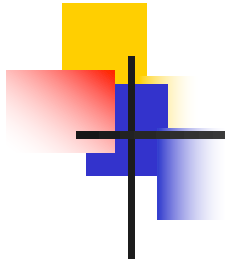
Pruebas automatizadas – Ejemplo

```
private User signUpUser(String userName) {  
  
    User user = new User(userName, "password", "firstName",  
        "lastName", userName + "@" + userName + ".com");  
  
    try {  
        userService.signUp(user);  
    } catch (DuplicateInstanceException e) {  
        throw new RuntimeException(e);  
    }  
  
    return user;  
  
}
```



Pruebas automatizadas – Ejemplo

```
private Category addCategory(String name) {  
    return categoryDao.save(new Category(name));  
}  
  
private Product addProduct(String name, Category category) {  
    return productDao.save(new Product(name, "description",  
        new BigDecimal(1), category));  
}  
  
private Product addProduct(String name) {  
    return addProduct(name, addCategory("category"));  
}  
  
...
```



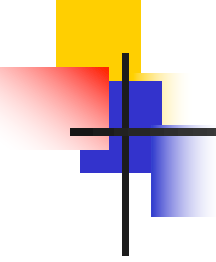
Pruebas automatizadas – Ejemplo

@Test

```
public void testBuyAndFindOrder() throws InstanceNotFoundException,
    PermissionException, MaxQuantityExceededException,
    MaxItemsExceededException, EmptyShoppingCartException {

    User user = signUpUser("user");
    Product product1 = addProduct("product1");
    Product product2 = addProduct("product2", product1.getCategory());
    int quantity1 = 1;
    int quantity2 = 2;
    String postalAddress = "Postal Address";
    String postalCode = "12345";

    shoppingService.addToShoppingCart(user.getId(),
        user.getShoppingCart().getId(), product1.getId(), quantity1);
    shoppingService.addToShoppingCart(user.getId(),
        user.getShoppingCart().getId(), product2.getId(), quantity2);
}
```



Pruebas automatizadas – Ejemplo

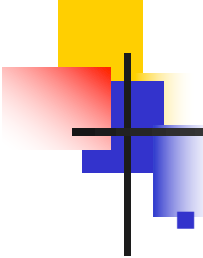
```
Order order = shoppingService.buy(user.getId(),
    user.getShoppingCart().getId(), postalAddress, postalCode);
Order foundOrder = shoppingService.findOrder(user.getId(),
    order.getId());

assertEquals(order, foundOrder);
assertEquals(user, order.getUser());
assertEquals(postalAddress, order.getPostalAddress());
assertEquals(postalCode, order.getPostalCode());
assertEquals(2, order.getItems().size());

Optional<OrderItem> item1 = order.getItem(product1.getId());
Optional<OrderItem> item2 = order.getItem(product2.getId());

assertTrue(item1.isPresent());
assertEquals(product1.getPrice(), item1.get().getProductPrice());
assertEquals(quantity1, item1.get().getQuantity());
assertTrue(item2.isPresent());
assertEquals(product2.getPrice(), item2.get().getProductPrice());
assertEquals(quantity2, item2.get().getQuantity());
assertTrue(user.getShoppingCart().isEmpty());

}
```



Pruebas automatizadas – Ejemplo

■ Obsérvese cómo otra vez se hace uso de métodos de negocio en las entidades para facilitar, esta vez, la implementación de las pruebas automatizadas

- `ShoppingCart.isEmpty` (ya usada anteriormente en `ShoppingServiceImpl.buy`)
- `Order.getItem`

■ En Order

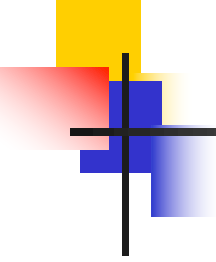
```
@Transient
public Optional<OrderItem> getItem(Long productId) {

    return items.stream().filter(
        item -> item.getProduct().getId().equals(productId) ).
        findFirst();

}
```

- **Pregunta:** ¿`ShoppingServiceTest.testBuyAndFindOrder` se ejecutaría con éxito si en `ShoppingServiceImpl.buy` se elimina `shoppingCart.removeAll()`?

- Pista: transaccionalidad, caché de primer nivel



Pruebas automatizadas – Ejemplo

@Test

```
public void testBuyEmptyShoppingCart() {  
  
    User user = signUpUser("user");  
  
    assertThrows (EmptyShoppingCartException.class, () ->  
  
        shoppingService.buy (user.getId() ,  
            user.getShoppingCart().getId() ,  
            "Postal Address", "12345"));  
  
}
```

- Existen múltiples estrategias sobre cómo redefinir **equals/hashCode** en las entidades
 - E.g. En el libro *Java Persistence with Hibernate* (bibliografía recomendada) se discute la problemática y se explica una estrategia
- Una posible estrategia consiste en observar que
 - [1] Cada caso de prueba se ejecuta en una transacción
 - [2] Dentro de la transacción, el mapeador usará caché de primer nivel => nunca cargará en memoria más de una instancia de una entidad con una misma clave primaria
- Consecuencia
 - Asumiendo que nuestro código no cree instancias duplicadas en memoria con la misma clave, no es necesario redefinir **equals/hashCode** (la igualdad referencial es suficiente)
 - Es la estrategia que se sigue en el caso de estudio

- En el caso de estudio se incluyen unas pocas pruebas automatizadas para algunos métodos de negocio de entidades que no se utilizan desde la capa Lógica de Negocio, sino desde la capa Servicios REST (como parte de la conversión de entidades a DTOs)
 - En consecuencia, estos métodos no son probados por las pruebas automatizadas de los servicios de la capa Lógica de Negocio
 - No requieren spring-test
 - **OrderTest**: prueba `getTotalPrice`
 - **ShoppingCartTest**: prueba `getTotalQuantity` y `getTotalPrice`